

Header, Scope, and Conventions

Web-Based Interactive Model Visualizer

Goal: Create a web-based Model visualizer that allows users to quickly change layers of the model as well as create models based on images uploaded via AI.

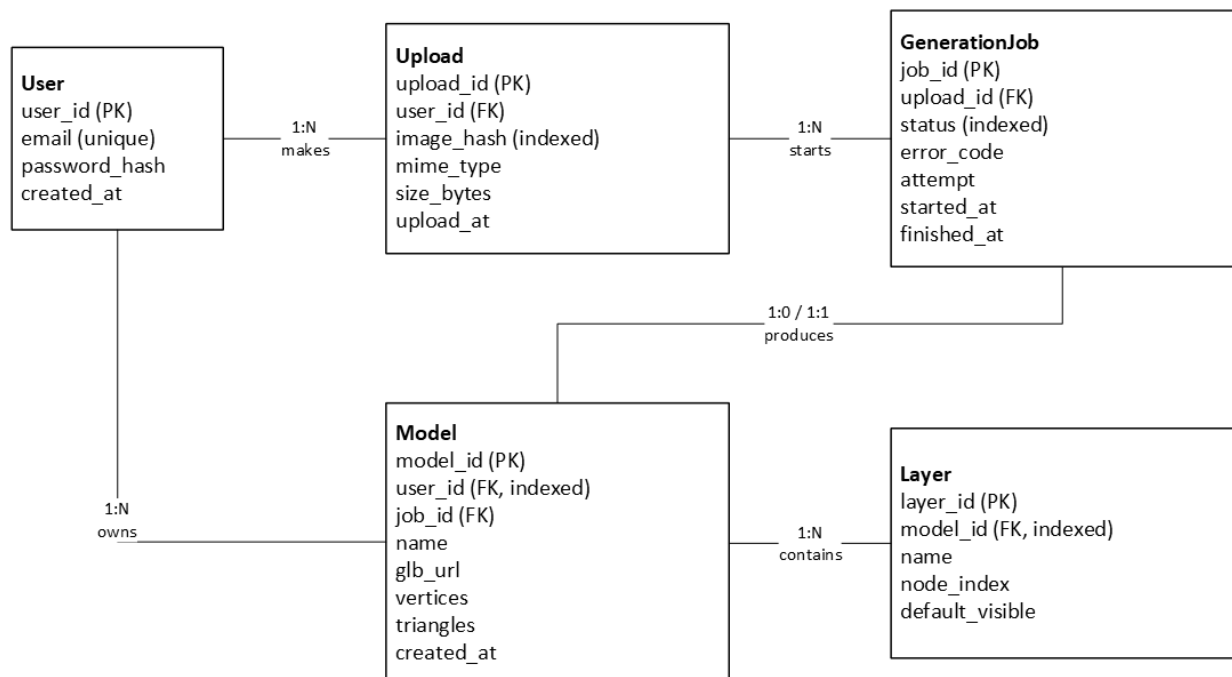
The D0 components that are in this document are:

- Image uploader

The components that the team deferred to D2 are:

- Login/Signup Section
- 3D Model Viewer
- Switcher (Main Screen)

Data Model, the D1 Diagram



Conventions:

Box: entity, name in header

PK: primary key

FK: foreign key

Line: relationship

Line label: cardinality

"indexed": field has database index

A user is able to make many uploads. Each upload can start several generation jobs. This is because a failed job can also be retried. A job can produce, when successful, at most one model. A user can own many models, and each of these models has many layers that the switcher and viewer can show/hide.

- **Table of Structural Decisions**

Decision	Why	Requirement
GenerationJob (entity), not a status flag on Upload	A job can have its own lifecycle, error code, and timestamps. An upload can have several jobs if a retry happens after a timeout. A status flag wouldn't be able to have that history.	UC-01 exception flow Algorithm 1 job records
Layer (entity), not a list stored inside Model	Each layer needs its own name and default visibility so the switch is able to toggle it. Layers are queried by model when viewer opens it.	US-01: brain cross-sections, US-02: transmission parts, US-03: rock layers, US—04: building floors
Upload-GenerationJob (one-to-many)	User can avoid re-uploading the image even if a failed generation occurs and needs to be retried	UC-01 exception flow
GenerationJob-Model (one-to-zero / one-to-one)	Only a successful job is able to create a model. A failed job creates none, so the model side is optional	Algorithm 2 GLB validation rejects bad files
User-Model (one-to-many)	Models are private to the user who created/imported them. There's no ability to share, so a join table would be unnecessary	Login/signup: individual user storage
GLB file located in file storage (db stores only glb_url)	GLB files can be several MB. Keeping them out of table rows keeps efficient queries, and url matches API (/api/v1/models/model_[1D])	API contract

Index for User.email (unique)	Login looks up a user by their email on every time they sign-in	login/signup
Index for Model.user_id	The switcher lists all of the user's models when they open the app	switcher
Index for Upload.image_hash	If someone reuploads the same image, the existing one can be returned instead of needing a new AI call	Algorithm 1 cost of AI
Index for GenerationJob.status	Finds jobs marked running after the timeout so they can be marked failed	Algorithm 1 timeout edge case
Index for Layer.model_id	Loads the layers for a model in one lookup when opened	viewer/switcher

We chose a relational model in PostgreSQL. The data is able to have clear entities with relationships that are queried together. A case that would call for such a model for example, is that job status needs consistency, meaning a model should not exist for a job that failed.

Core Algorithms

Algorithm 1: Image-to-GLB generation pipeline

- **What is it and the Problem it Solves:**
 - This will take the image that the user uploads, will have a check that it is usable, and then will get a 3D model generated from it by the AI backing service. The result will then be checked that it is a valid GLB and will be stored in a database so that it doesn't need to be regenerated. This solves the problem of accessibility of obtaining a trustworthy model from a possible untrustworthy input and done so at a normal rate, while keeping the cost of AI down.
- **Inputs and Outputs with Exact Types:**
 - Input:
 - Name: image
 - Type: image file, png, jpeg
 - Input:
 - Name: userId
 - Type: integer

- Output:
 - Name: modelId
 - Type: integer
- Output:
 - Name: GLB
 - Type: GLB file, gltf-binary
- Output (for error):
 - Name: errorCode
 - Type: enum
- **Expected Complexity at your Realistic Data Size and at 100x:**
 - The validation and transfer are linear in their file size, so it would be $O(n)$ and does not compare to the AI call.
 - The AI call will have a cost per request and with a measurable latency which can depend on user network, but with server hosting it is typical with a delay of 2-3 seconds with a fast time to first token of under 300-500ms. The cost per request depends on what model and what tier of model you are using but for standard API calls the cost ranges from \$0.001 to \$0.05.
 - The storage lookup for users and their already generated files could be $O(n \log n)$.
 - When comparing a 100x time complexity, the main changes are concurrency, polling traffic, storage, and AI cost. The issue is not much with algorithmic time.
- **Why this Approach Rather than the Alternatives:**
 - With our approach it is better for the performance, cost, and scalability, where with our approach the only need is the HTTP calls which will be done in a client-server work, where alternatives could need a GPU setup with model serving. The cost of paying per generating is cheaper than GPU hosting. Also, the scalability is better as we would just have to manage rate limits and not have to scale GPUs ourselves.
- **Edge Cases:**
 - The model failed to load.
 - Empty file or zero bytes.
 - Incorrect format uploaded
 - Image size is out of scope for what can be handled.
 - The image is uploaded with no object, so AI can't make it out.
 - AI service times out.
 - AI returns invalid/empty GLB.
 - Tabs close mid file generation.

- Storage fails after a successful generation.
- Limit is hit for uploads.
- **Steps, Clear Prose a Teammate could Implement:**
 - If the user is not authenticated, the request would be rejected.
 - File type is checked by reading the files header bytes then size dimensions. If any does not fit into the regulations, then it will be rejected.
 - There could be an option to downscale images that are too large or downscale to cut transfer time.
 - Job records created with status of jobs as image gets sent to AI service.
 - The user waits for results up to a timeout where job status would be updated as it goes.
 - Validate returned file and failure status is failed, and an error code is returned.
 - GLB and metadata are saved to the specific database, modelId is returned to the switcher.

Algorithm 2: GLB Validation

- **What is it and the Problem it Solves:**
 - It will check that the file returned by the AI service is in the right GLB format so that it can be sent to the switcher, and the viewer can load it before storing.
 -
- **Inputs and Outputs with Exact Types:**
 - Input:
 - Name: glbBytes
 - Type: GLB file, gltf-binary
 - Output:
 - Name: validImage
 - Type: boolean
 - Output (for invalid):
 - Name: reason
 - Type: enum
- **Expected Complexity at your Realistic Data Size and at 100x:**
 - Checking the header is $O(1)$ and parsing the JSON depends on the size of the file outputted chunk size.
 - Full validation would be $O(n)$.
 - At 100x the only thing that grows is the concurrency since the file size is not changing with the number of users using the service.

- **Why this Approach Rather than the Alternatives:**
 - With header and JSON checks and a reused validator will catch failures quickly and cheaply. While only checking extensions can miss corrupted files.
- **Edge Cases:**
 - Empty response.
 - Truncated file.
 - Wrong version.
 - Missing binary chunks.
 - Corrupt JSON chunk.
 - File over size.
- **Steps, Clear Prose a Teammate could Implement:**
 - If the file is empty or shorter than the 12-byte header, it is rejected.
 - First, check the header and the first four bytes, the version, and the declared length equals the actual file length.
 - Reject if the file exceeds the size cap.
 - First chunk is JSON check then parsed. JSON must be valid with an asset field.
 - Run the reused g1TF validator for a deeper check, which will return valid or failing reason.

Build-Versus-Reuse Decisions

Detailed Components	Build or Reuse	
Upload button	build	This component has specific logic to our site.
File upload	Reuse	This is a common component that can be found online, will probably use one of the one on this site(https://ant.design/components/upload) based upon which one looks best on the site
Upload API	build	This can be based on existing examples, but we will need to do some of our own processing to get it to the AI.
AI model generation	reuse	Making an entire 3d modeling AI would be far outside the scope of this project. We might have to give the AI some specific prompts, but it will be the most up to date ChatGPT model.

3d model saving	build	Like the upload to the api some of this can probably be based off existing examples but we will need some specific logic to get it to the
-----------------	-------	---

API Contract

Image Upload HTTP – POST request:

POST parameters:

Name	Type	Required?	Valid Range
File	File	Yes	PNG or JPEG (max size = 20Mb)
Name	string	Yes	1-255 characters
UserID	int	Yes	0 - 2,147,483,647 (int32)
Resolution	int	No	256 – 4096 pixels

Output GLB file:

JSON

{

ModelID = 1 (primary key, unique key)

Name: “Example Name” (name the user wants to store the model under)

Format: “glb” (always will be a glb)

URL: url for the model

Vertices: # of vertices in model

Triangles: # of triangles in model

Generated: 0 or 1 bit, 1 if generated as expected 0 if generation failed,

Error: message if there was an error in the backend

}

Error Codes:

Status	What happens
400 Invalid Request	Lack of parameters
413 File too large	Image file exceeds 20Mb limit
415 Unsupported File	User uploaded a file not PNG or JPEG (should be handled in front end but will include back-end handling as well)
422 Generation failed	3D generation fails to build
500 Server error	Unhandled server error

Request:

POST

File: example_model.jpeg

Name: "Test Example"

UserID: 2

Resolution: 2048

Response

JSON

```
{  
  ModelID = 2,  
  Name: "Test Example",  
  Format: "glb",  
  URL: "/api/v1/models/model_2",  
  Vertices: 45218,  
  Triangles: 87342,  
  Generated: 1  
  Error: ""  
}
```

Versioning will be handled through the file URL, each model will be named with their unique ID to allow for numerous files to be uploaded and stored without issues on the server. The ID will be a unique integer and the url will end with “model_[ModelID]”.

Technology Choices with Justification

Error/Reupload - Backend error handling:

For error handling, we will have various error handling messages based on the various scenarios we end up coming up with during the development phase. In the event of an error such as invalid credentials or any sort of issue converting the image to a 3d model will display the error message to the user and then prompt them to reupload a valid image into the system. Upon upload, we will probably have a front-end error handling to check the file type (png or jpeg) and prompt the user to reupload there. This is to limit the amount of unnecessary http requests that will be sent to the backend/server.

Store uploaded file to database – database:

For storing the uploaded and generated models, we chose a relational database, PostgreSQL to hold users, model records, and job status. PostgreSQL uses an open-source license and has a large established community with extensive documentation so if there is any issue along the way, we should be able to find a solution to answer our questions. For performance, our queries will be indexed lookups which stay as fast as the table grows. On cost and hosting, PostgreSQL is free to run, and we plan to host it either on a cheap serverless platform like Railway or Neon, or on a traditional cloud server environment like AWS or Microsoft Azure.

Custom file uploader vs prebuilt image uploader:

The file uploader is an integral part of the design as well as just being something that naturally takes up a good portion of the screen. This made it a difficult decision to decide if it should be prebuilt or if it should be built by us. Building it ourselves would give us full control over not only how it looked but also how it acted. Building it ourselves would also take a great deal of effort in an already large project. A prebuilt file uploader would greatly simplify a major part of the project while still giving us a pretty good look. There is also a wide range of uploaders for us to pick from allowing us to choose one that might look better than what we could make. Given the advantages of a prebuilt we decided to go with a prebuilt component.

Image uploading GUI look:

For the image uploading GUI, for the moment the front end will be React with Ant Design's Upload component. The team decided on React with Ant Design over building a custom drag-and-drop uploader on a plain HTML file input. A couple of our team members have experience using React in personal projects. Both React and Ant Design are under MIT licenses. This license would allow the team to use and modify without any copyright issues due to the project being a university project. Ant Design is a widely used component library maintained in React. It has documentation that provides various solutions to drag-and-drop, progress bars, and file preview functionality. As for the performance, Ant's upload component has file type and size checkers in the browser before anything is sent, so this avoids unnecessary requests for the backend. Both resources are open source, and the team will be able to fully implement and host our project in Vercel or GitHub Pages.